

MyLENS — Developer Handover Guide

For engineers taking over this project

English · Easy to follow

Developer overview

This guide is for a **new developer** taking over MyLENS.

Read it first. Keep language simple. Follow the linked pages in order.

What you are inheriting

Item	Detail
Product	MyLENS — youth tourism video campaign + learning academy
App type	Next.js web app (App Router)
UI	React 19 + Tailwind CSS 4
Backend	Supabase (Auth, Postgres, Storage, RLS)
Language	TypeScript
Package manager	npm

What the product does

1. **Public website** — campaign story, schools map, videos, contact / partnership
2. **Creator workspace** — lessons, homework tasks, contest video submissions, team, profile
3. **Admin workspace** — schools, people approval, video review, learning content, inquiries

Important product rule (do not mix these)

Feature	Tables / area	UI
Contest video	submissions	Admin Submissions / Creator Submissions
Learning homework	learning_task_submissions	Admin Learning → Reviews / Creator My tasks
Learning lessons	learning_modules, learning_items	Admin Lessons / Creator Lessons

If a bug report says “submission”, ask: **contest video** or **homework task**?

Repository layout (important folders)

```
mylens-2026/
  app/                Next.js routes (auth, public, dashboards)
```

components/	UI (admin/, creator/, sections/, auth/)
lib/	Server actions, queries, auth, supabase clients
types/	Shared TypeScript types
supabase/migrations/	SQL migrations (run in order)
supabase/README.md	Setup notes for Supabase
(UserManual & Guide)/	User + developer docs + PDF build
.env.local	Local secrets (do not commit)
package.json	Scripts and dependencies
AGENTS.md	Note: read Next.js docs in node_modules before big changes

Roles

Stored on `public.profiles.role` (and related auth metadata):

Role	Goes to
creator	/dashboard/creator
admin	/dashboard/admin

Creators often start as **pending** until an admin activates them.

There is **no judge workspace** in the current product. Do not create judge accounts.

Documentation map for developers

File	Purpose
01 setup and run.md	Install, env, run locally
02 supabase and database.md	Migrations, tables, storage
03 architecture.md	How the app is structured
04 features and key files.md	Where each feature lives
05 deploy and auth.md	Hosting + Auth redirect URLs
06 handover checklist.md	Day-1 checklist

Also skim the **user** guides so you understand the product screens.

Tech notes before you code

1. This project uses a **newer Next.js** than many tutorials. Read `AGENTS.md` and `node_modules/next/dist/docs/` when APIs look unfamiliar.
2. Prefer existing patterns in `lib/` and `components/` over inventing new folders.
3. Database rules (RLS) control what creators see. “Missing content” is often **draft / unpublished**, not a React bug.
4. Never commit real service-role keys.

Setup and run

This page helps you run MyLENS on your computer.

What you need

- Node.js (LTS recommended)
 - npm (comes with Node)
 - Git access to this repository
 - Access to the Supabase project used by the team
 - A code editor (Cursor / VS Code)
-

1. Get the code

```
cd "D:\Android Studio Project\mylens-2026"
```

If you clone fresh, open that folder as the project root.

2. Install packages

```
npm install
```

3. Create `.env.local`

In the project root, create or edit `.env.local` :

```
NEXT_PUBLIC_SUPABASE_URL=your-supabase-url  
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key
```

Optional (admin scripts only — keep secret):

```
SUPABASE_SERVICE_ROLE_KEY=your-service-role-key
```

Where to find the values

Supabase Dashboard → **Project Settings** → **API**

- Project URL → `NEXT_PUBLIC_SUPABASE_URL`
- `anon` `public` key → `NEXT_PUBLIC_SUPABASE_ANON_KEY`

Restart the dev server after changing env values.

4. Confirm database migrations

Ask the team if migrations `001` through `014` are already applied on the shared Supabase project.

If you are setting up a **new** empty project, run the SQL files in order.
See [02 supabase and database.md](#).

5. Start the app

```
npm run dev
```

Open: <http://localhost:3000>

Useful npm scripts

Command	What it does
npm run dev	Local development
npm run build	Production build
npm run start	Run production build locally
npm run lint	Lint check

Quick smoke test after setup

1. Public home page loads
 2. Login page opens
 3. Creator account can open `/dashboard/creator` (if approved)
 4. Admin account can open `/dashboard/admin`
 5. Learning pages load without server errors
-

Common setup problems

Blank errors talking to Supabase

- Wrong URL or anon key
- Dev server not restarted after `.env.local` change
- You are pointed at a different Supabase project than the one with data

Login works but dashboard fails

- Profile row missing / pending
- RLS issue (see migration `006_fix_profiles_rls_recursion.sql`)
- Role mismatch

Learning tasks missing for creators

- Migration `014_learning_tasks.sql` not applied, or
 - Module / item still draft (not published)
-

Do not do this

- Do not commit `.env.local`
- Do not paste service role keys into client code
- Do not run random SQL out of order on production without a backup plan

Supabase and database

This page explains the backend for MyLENS in plain language.

What Supabase is used for

Part	Used for
Auth	Login, register, password reset
Postgres	Schools, profiles, submissions, learning, inquiries
Storage	Learning files, task uploads, related media
RLS	Who can read / write each row

Migrations (run in order)

Files live in `supabase/migrations/`.

Order	File	Purpose
001	<code>001_phase0_foundation.sql</code>	Base foundation
002	<code>002_registration.sql</code>	Creator registration
003	<code>003_school_access_tokens.sql</code>	School access tokens
004	<code>004_creator_limit.sql</code>	Creator limit per school
005	<code>005_phase3_creator_portal.sql</code>	Creator portal pieces
006	<code>006_fix_profiles_rls_recursion.sql</code>	Fix profile RLS recursion
007	<code>007_phase4_submissions.sql</code>	Contest video submissions
008	<code>008_creator_limit_four.sql</code>	Creator limit update
009	<code>009_public_map_stats.sql</code>	Public map stats
010	<code>010_contact_inquiries.sql</code>	Contact inquiries
011	<code>011_partnership_inquiries.sql</code>	Partnership inquiries
012	<code>012_learning_hub.sql</code>	Learning modules + items
013	<code>013_learning_progress.sql</code>	Lesson completion progress
014	<code>014_learning_tasks.sql</code>	Homework tasks + task submissions

015	015_remove_judge_access.sql	Drop unused judge school policy
-----	-----------------------------	---------------------------------

Also read: [supabase/README.md](#)

How to apply a migration

1. Open Supabase Dashboard → **SQL** → New query
2. Paste the migration file contents
3. Run
4. Fix errors before continuing to the next file

For an existing team project, confirm with the owner which files are already applied.

Main tables (friendly map)

Table	Meaning
<code>schools</code>	Participating schools
<code>profiles</code>	Users (role, status, school)
<code>submissions</code>	Contest tourism videos
<code>campaign_settings</code>	Submission open / close times
<code>campaign_resources</code>	Campaign resource links
<code>learning_modules</code>	Learning folders
<code>learning_items</code>	Lessons + assignments (<code>content_type</code> includes task)
<code>learning_item_progress</code>	Creator lesson completion
<code>learning_task_submissions</code>	Homework answers for review
<code>contact_inquiries</code>	School contact form messages
<code>partnership_inquiries</code>	Partnership form messages

Learning publish rules (backend + product)

Creators can see a lesson/assignment only when:

1. `learning_items.published = true`
2. Parent `learning_modules.published = true`
3. Creator can access learning (active creator rules)

Admin can see drafts. Creators cannot.

Storage buckets to know

Bucket	Used for
--------	----------

learning-content	Lesson videos / documents uploaded by admin
learning-tasks	Creator homework file uploads (from migration 014)

Confirm buckets exist in Supabase → **Storage**.

Useful checks in SQL Editor

Is this assignment visible to creators?

```
select
  m.title as module,
  m.published as module_published,
  i.title as item,
  i.content_type,
  i.published as item_published
from learning_items i
join learning_modules m on m.id = i.module_id
where i.content_type = 'task'
order by i.created_at desc;
```

Promote / check an admin (example pattern)

Only do this with team approval. Exact SQL may depend on your profile setup. Prefer using the admin UI when possible.

Seed / test users

supabase/README.md describes example test users and school tokens for local/dev setups.

Do not use production passwords in docs or chat logs.

Care points

1. Missing migration = features "half work" in the UI
 2. RLS recursion on profiles → run 006 if login cannot load profile
 3. Task homework needs 014 + learning-tasks bucket
 4. Contest videos and learning tasks are different tables
-

Architecture

This page explains how the MyLENS app is organized.

Keep changes small and follow existing patterns.

Big picture

```
Browser (Next.js app)
↓
Server components / server actions / route handlers
↓
Supabase client (user session) or service role (rare admin scripts)
↓
Postgres + Storage + Auth (with RLS)
```

App Router folders

Area	Path	Purpose
Public site	app/page.tsx, app/schools, app/videos	Marketing / public pages
Auth	app/(auth)/login, register, forgot-password	Sign in flows
Shared dashboard shell	app/(dashboard)/layout.tsx	Logged-in chrome
Creator	app/(dashboard)/dashboard/creator/...	Student workspace
Admin	app/(dashboard)/dashboard/admin/...	Staff workspace

Auth and roles

Key files:

- `lib/auth/session.ts` — require login / role
- `lib/auth/roles.ts` — role helpers
- `lib/supabase/client.ts` — browser client
- `lib/supabase/server.ts` — server client
- `lib/supabase/middleware.ts` — session refresh / protection helpers

Pattern used in pages:

1. `requireRole(["creator"])` or `requireRole(["admin"])`
2. Load data with server queries
3. Render UI components

Data flow pattern (preferred)

For dashboard features, the common pattern is:

1. **Page** (server) loads data from `lib/.../queries.ts`
2. **Component** shows UI
3. **Server action** in `lib/.../actions.ts` writes changes
4. `revalidatePath(...)` refreshes the related screens

Example learning areas:

- Queries: `lib/learning/queries.ts`
- Lesson actions: `lib/learning/actions.ts`

- Task actions: `lib/learning/task-actions.ts`

UI layers

Folder	Use for
<code>components/admin/</code>	Admin-only UI + shared admin design tokens (<code>AdminUI.tsx</code>)
<code>components/creator/</code>	Creator workspace UI
<code>components/sections/</code>	Public landing sections
<code>components/auth/</code>	Login / register forms
<code>types/</code>	Shared types (auth, learning, submission, ...)

Design note: admin learning uses forest/brass tokens in `components/admin/AdminUI.tsx` .

Learning architecture (easy map)

```
learning_modules
├─ learning_items
│   ├── content_type = live_class / recorded_video / document / external_link
│   └─ content_type = task
│       └─ learning_task_submissions (per creator)
```

Creator progress for normal lessons uses `learning_item_progress` .

Task completion is tied to mentor **approval**, not self “mark complete”.

Contest submissions architecture

```
profiles + schools
├─ submissions (video entries)
│   └─ admin moderation notes / status
```

Campaign timing comes from `campaign_settings` .

Forms and safety

- Validate on the server in actions
 - Keep dangerous operations behind `requireRole(["admin"])`
 - Confirm destructive UI actions in the browser when needed
 - Prefer clear status enums already defined in `types/`
-

Next.js caution

This repo notes that Next.js APIs may differ from older training data.

Before large framework changes:

- 1. Read AGENTS.md
- 2. Check docs under node_modules/next/dist/docs/

When adding a feature

Ask:

- 1. Which role uses it?
- 2. Which table owns the data?
- 3. Is there already a query/action file for this domain?
- 4. Which paths must be revalidated?
- 5. Does RLS already allow the action?

Features and key files

Use this page to find the right files quickly.

Auth

Feature	Key places
Login	components/auth/LoginForm.tsx, app/(auth)/login
Register	components/auth/RegisterForm.tsx, app/(auth)/register
Forgot password	components/auth/ForgotPasswordForm.tsx
Session / roles	lib/auth/session.ts, lib/auth/roles.ts

Public website

Feature	Key places
Landing sections	components/sections/*, app/page.tsx
Schools page	app/schools
Videos page	app/videos
Contact inquiry	lib/contact/actions.ts
Partnership inquiry	lib/partnership/actions.ts, components/partnership/*

Creator workspace

Feature	Route	Key places
---------	-------	------------

Home	/dashboard/creator	app/.../creator/page.tsx, CreatorOverview, CreatorNextStep
Lessons	/dashboard/creator/learning	LearningHub.tsx, lib/learning/queries.ts
My tasks	/dashboard/creator/learning/tasks	CreatorTasksHub.tsx, LearningTaskSubmitForm.tsx, lib/learning/task-actions.ts
Submissions list	/dashboard/creator/submissions	SubmissionsWorkspace.tsx
New submission	/dashboard/creator/submissions/new	SubmissionForm.tsx, lib/creator/submission-actions.ts
Team	/dashboard/creator/team	team components + creator queries
Profile	/dashboard/creator/profile	ProfileForm.tsx
Nav	—	components/creator/CreatorNav.tsx

Admin workspace

Feature	Route	Key places
Overview	/dashboard/admin	AdminAttentionPanel, stats
Contest submissions	/dashboard/admin/submissions	lib/admin/submissions.ts, moderation form
Submission detail	/dashboard/admin/submissions/[id]	SubmissionModerationForm.tsx
Inquiries	/dashboard/admin/inquiries	AdminInquiriesHub.tsx
People	/dashboard/admin/users, .../pending	user status actions
Schools	/dashboard/admin/schools	school forms, tokens
Lessons	/dashboard/admin/learning	LearningAdminPanel mode lessons
Assignments	/dashboard/admin/learning/assignments	LearningAdminPanel mode assignments
Task reviews	/dashboard/admin/learning/tasks	task list + moderation
Nav	—	components/admin/AdminNav.tsx
Design tokens	—	components/admin/AdminUI.tsx

Learning domain (shared)

Concern	File
Types / labels	types/learning.ts
Queries	lib/learning/queries.ts

Module/lesson CRUD + complete	lib/learning/actions.ts
Task submit + moderate	lib/learning/task-actions.ts
Task helpers / filters	lib/learning/tasks.ts
SQL	012, 013, 014 migrations

Contest submissions domain

Concern	File
Types	types/submission.ts, types/auth.ts
Creator write	lib/creator/submission-actions.ts
Admin review	lib/admin/submission-actions.ts
Admin list helpers	lib/admin/submissions.ts
SQL	007_phase4_submissions.sql

Schools and people

Concern	File
School admin logic	lib/admin/schools.ts, lib/admin/tokens.ts
Admin actions	lib/admin/actions.ts
Creator context	lib/creator/queries.ts

User manuals / docs build

Concern	File
Docs home	(UserManual & Guide)/README.md
User chapters	(UserManual & Guide)/user/*
Developer chapters	(UserManual & Guide)/developer/*
PDF builder	(UserManual & Guide)/scripts/build-pdf-manuals.mjs

Rebuild:

```
node "(UserManual & Guide)/scripts/build-pdf-manuals.mjs"
```

Debugging tip

If UI looks fine but data is wrong:

1. Check role + profile status
2. Check published flags (learning)
3. Check the correct table (video vs task)
4. Check whether migration exists on the active Supabase project
5. Check browser network / server action errors

Deploy and auth

This page covers hosting and login URL settings.

Typical hosting

MyLENS is a Next.js app. Teams often deploy on **Vercel**.

High-level steps:

1. Connect the Git repository to the host
 2. Set environment variables
 3. Deploy
 4. Update Supabase Auth URLs to match production
-

Environment variables on the host

Set at least:

Name	Notes
NEXT_PUBLIC_SUPABASE_URL	Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Public anon key

Optional:

Name	Notes
SUPABASE_SERVICE_ROLE_KEY	Server-only secret. Never expose to the browser.

Use the **same Supabase project** the team expects for that environment (staging vs production).

Supabase Auth URLs

In Supabase Dashboard → **Authentication** → **URL configuration**:

Local development example

- Site URL: `http://localhost:3000`
- Redirect URLs include:
 - `http://localhost:3000/api/auth/callback`
 - other local auth callback paths your app uses

Production example

- Site URL: your live domain (example `https://your-domain.com`)
- Redirect URLs include the production callback URL(s)

If password reset opens the wrong page, Redirect URLs are usually wrong.

Email settings (dev vs production)

For local testing, teams sometimes disable **Confirm email** so registration is faster. Admin approval of creator profiles may still be required.

For production:

- Use a proper email provider setup in Supabase when needed
 - Watch rate limits on built-in email during heavy testing
-

After deploy checklist

- ☐ Home page loads on production URL
 - ☐ Login works
 - ☐ Forgot password email link returns to the correct domain
 - ☐ Creator dashboard loads for an approved creator
 - ☐ Admin dashboard loads for an admin
 - ☐ File upload buckets work (learning content / learning tasks)
 - ☐ Contest submission create works when window is open
-

Common deploy mistakes

1. Env vars missing on the host
 2. Deployed app pointed at the wrong Supabase project
 3. Auth redirect still set to localhost only
 4. Migrations applied on staging but not production
 5. Storage policies/buckets missing on production
-

Security basics

- Never commit `.env.local`
 - Never put service role key in client components
 - Prefer server actions / server components for privileged writes
 - Review RLS before opening new tables to authenticated users
-

Handover checklist

Use this on day 1 of taking over MyLENS.

Access you should receive

- ☐ Git repository access
 - ☐ Supabase project access (Dashboard)
 - ☐ Hosting access (for example Vercel)
 - ☐ Production / staging URLs
 - ☐ At least one admin test account
 - ☐ At least one approved creator test account
 - ☐ This documentation folder
-

Verify the environment

- ☐ `npm install` works
 - ☐ `.env.local` has correct Supabase URL + anon key
 - ☐ `npm run dev` opens <http://localhost:3000>
 - ☐ App talks to the expected Supabase project
 - ☐ Migrations through `014_learning_tasks.sql` are applied (or you know what is missing)
 - ☐ Storage buckets exist (`learning-content` , `learning-tasks`)
 - ☐ Auth Site URL + Redirect URLs match local and production needs
-

Verify product flows

Public

- ☐ Home page loads
- ☐ Schools / videos pages open
- ☐ Contact or partnership form can submit (if enabled)

Creator

- ☐ Register / login
- ☐ Pending creator cannot fully work until approved
- ☐ Approved creator opens Home
- ☐ Lessons open under Learning → Lessons
- ☐ My tasks open under Learning → My tasks
- ☐ Contest Submissions list opens
- ☐ Team and Profile open

Admin

- ☐ Overview loads
 - ☐ Approve a pending creator
 - ☐ Open Schools
 - ☐ Open contest Submissions and open one detail page
 - ☐ Learning → Lessons catalogue loads
 - ☐ Learning → Assignments catalogue loads
 - ☐ Learning → Reviews queue loads
 - ☐ Inquiries page loads
-

Read these docs

- ☐ [\(UserManual & Guide\)/README.md](#)
 - ☐ [User getting started](#)
 - ☐ [Creator guide](#)
 - ☐ [Admin guide](#)
 - ☐ [Developer overview](#)
 - ☐ [Setup and run](#)
 - ☐ [Supabase and database](#)
 - ☐ [Architecture](#)
 - ☐ [Features and key files](#)
 - ☐ [Deploy and auth](#)
-

Known care points (do not ignore)

1. **Wrong Supabase project** — many “bugs” are missing SQL on the active project
 2. **Draft learning content** — creators only see published module + published item
 3. **Video vs homework** — `submissions` $\neq$ `learning_task_submissions`
 4. **Migration 014** — required for assignments / My tasks / Reviews
 5. **Profile RLS** — if profile cannot load after older migrations, check `006`
 6. **Next.js version** — read `AGENTS.md` before assuming old Next APIs
 7. **Auth redirect URLs** — required for password reset to work on each domain
-

When you change a feature

1. Update or add a migration if the database changes
2. Update user docs if screens or button names change
3. Rebuild PDFs:

```
node "(UserManual & Guide)/scripts/build-pdf-manuals.mjs"
```

4. Test both creator and admin paths when the change touches Learning or Submissions
-

First-week goal

Be able to:

- Run the app locally
- Explain video submissions vs learning tasks
- Publish a lesson and an assignment
- Review one contest video and one homework task
- Know where env, migrations, and key feature files live